```
# meme_bot.py - PAPER-ONLY Solana memecoin bot. PRETEND MONEY. It never touches a
# It reads public data (GeckoTerminal, RugCheck), decides with fixed rules, and l
# Files: meme_state.json, meme_trades.csv, meme_equity.csv, meme_funnel.csv, meme
import csv
import json
import math
import os
import subprocess
import time
from datetime import datetime, timedelta, timezone

import requests

START_CASH = 1000.0
SIZE_PCT = 0.08            # each position is 8% of the account
MAX_POSITIONS = 5
MAX_BUYS_PER_RUN = 2
STOP_PCT, TRAIL_START_PCT, TRAIL_DROP_PCT, MAX_HOURS = 6.0, 12.0, 10.0, 24
MIN_LIQ, MIN_H1_VOL = 30_000, 40_000
MIN_AGE_MIN, MAX_AGE_H = 30, 12
MIN_FDV, MAX_FDV = 100_000, 20_000_000
MIN_H1_CHG, MAX_H1_CHG = 5.0, 120.0
MIN_BUYERS, MIN_SELLERS, MIN_BUY_SELL_RATIO = 60, 20, 1.1
RUG_LIQ_FRACTION = 0.30     # liquidity falling below 30% of entry = rug
NEW_PAGES = 5               # newest-pool pages scanned per run (20 pools each)
WATCH_MAX = 400             # coins remembered while they mature
REFRESH_PER_RUN = 120       # remembered coins re-checked per run
WRITE_OFF_AFTER_MISSING = 8 # runs without data (about 2 hours) = total loss
FEE_PCT = 0.5               # DEX fee per side
BASE_SLIP_PCT = 1.0         # minimum slippage per side
NETWORK_FEE_USD = 0.25      # priority fee / tip per trade
GT = "https://api.geckoterminal.com/api/v2"
SOL_MINT = "So11111111111111111111111111111111111111112"
STATE, TRADES, EQUITY, FUNNEL, PROBE = "meme_state.json", "meme_trades.csv", "mem
SHADOW = "meme_shadow.csv"
SHADOW_ALL = "meme_shadow_all.csv"   # wide log: every coin that looks real enoug
SHADOW_ALL_MAX = 400
ALL_REG_PER_RUN = 15
ALL_MIN_LIQ, ALL_MIN_VOL = 10_000, 10_000
CHECKPOINTS_H = (1, 4, 24)    # shadow log: look again this many hours after a co
SHADOW_MAX = 300
HEADERS = {"Accept": "application/json;version=20230302", "User-Agent": "paper-me
```

```
def num(x):
    try:
        v = float(x)
        return None if math.isnan(v) else v
    except (TypeError, ValueError):
        return None


def http_json(url):
    try:
        r = requests.get(url, headers=HEADERS, timeout=20)
        if r.status_code != 200:
            print(f"data: {r.status_code} from {url.split('?')[0][:90]}")
            return None
        return r.json()
    except Exception as e:
        print(f"data: failed {url.split('?')[0][:90]} ({e})")
        return None


def append_csv(path, header, row):
    new = not os.path.exists(path)
    with open(path, "a", newline="") as f:
        w = csv.writer(f)
        if new:
            w.writerow(header)
        w.writerow(row)


def load_state():
    try:
        with open(STATE) as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return None


def parse_pool(item):
    """One GeckoTerminal pool -> flat dict, or None if it can't be read."""
    try:
        a = item["attributes"]
        base = (item.get("relationships", {}).get("base_token", {}).get("data", {
        h1 = (a.get("transactions") or {}).get("h1") or {}
        return {"pool": a["address"], "name": a.get("name", "?"), "mint": base.sp
                "price": num(a.get("base_token_price_usd")), "liq": num(a.get("re
                "vol1h": num((a.get("volume_usd") or {}).get("h1")), "chg1h": num
                "buys": num(h1.get("buys")), "sells": num(h1.get("sells")), "buye
```

```
                "created": a.get("pool_created_at")}
    except Exception:
        return None


def age_minutes(created, now):
    try:
        t = datetime.fromisoformat(str(created).replace("Z", "+00:00"))
        return (now - t).total_seconds() / 60
    except Exception:
        return None


def rug_ok(mint, get=http_json):
    """RugCheck summary. Fail closed: anything unreadable counts as NOT ok."""
    if not mint:
        return False, "no mint"
    d = get(f"https://api.rugcheck.xyz/v1/tokens/{mint}/report/summary")
    if not isinstance(d, dict) or "risks" not in d or not isinstance(d["risks"],
        return False, "unreadable"
    bad = [str(r.get("name", "?")) for r in d["risks"] if isinstance(r, dict) and
    return (not bad), (", ".join(bad) if bad else "clean")


def cost_pct(size_usd, liq):
    impact = (size_usd / (liq / 2)) * 100 if liq else 100.0
    return FEE_PCT + max(BASE_SLIP_PCT, 2 * impact)


def equity(s, prices):
    return s["cash"] + sum(p["qty"] * prices.get(k, p.get("last", p["avg"])) for


def log_trade(now, coin, side, qty, price, fee, usd, rule, why):
    append_csv(TRADES, ["time_utc", "coin", "side", "qty", "price", "fee_usd", "u
               [now.isoformat(timespec="seconds"), coin, side, f"{qty:.8g}", f"{p


def sell(s, key, price, liq, now, rule, why):
    p = s["positions"].pop(key)
    gross = p["qty"] * price
    pct = cost_pct(gross, liq) if price > 0 else 0
    fee = gross * pct / 100 + (NETWORK_FEE_USD if price > 0 else 0)
    s["cash"] += max(gross - fee, 0)
    s["cooldown"].append(key)
    pnl = (price / p["avg"] - 1) * 100 if p["avg"] else 0
    log_trade(now, p["name"], "sell", p["qty"], price, fee, max(gross - fee, 0),
```

```
def fetch_candidates(get=http_json):
    pools, answered = {}, False
    urls = [f"{GT}/networks/solana/new_pools?page={k}" for k in range(1, NEW_PAGE
    raw_first = None
    for u in urls:
        d = get(u)
        if not isinstance(d, dict):
            continue
        answered = True
        for it in d.get("data", []) or []:
            if raw_first is None:
                raw_first = it
            p = parse_pool(it)
            if p and p["price"]:
                pools[p["pool"]] = p
        time.sleep(2.2)
    return pools, raw_first, answered


def fetch_held(addrs, get=http_json):
    out = {}
    for k in range(0, len(addrs), 30):
        chunk = addrs[k:k + 30]
        d = get(f"{GT}/networks/solana/pools/multi/{','.join(chunk)}")
        if isinstance(d, dict):
            for it in d.get("data", []) or []:
                p = parse_pool(it)
                if p and p["price"]:
                    out[p["pool"]] = p
        time.sleep(2.2)
    return out


def sol_price(get=http_json):
    d = get(f"{GT}/simple/networks/solana/token_price/{SOL_MINT}")
    try:
        return num(d["data"]["attributes"]["token_prices"][SOL_MINT])
    except Exception:
        return None


def check_exits(s, held, cands, prices, now, count_missing=True):
    """Stop loss, trailing stop, rug and time limit, using the prices we can actu
    for key in list(s["positions"]):
        p = s["positions"][key]
```

```
        cur = held.get(key) or cands.get(key)
        if not cur:
            if not count_missing:
                continue          # fast checks never write a coin off: one misse
            p["missing"] = p.get("missing", 0) + 1
            if p["missing"] >= WRITE_OFF_AFTER_MISSING:
                sell(s, key, 0.0, 0, now, "vanished", "no data for about 2 hours,
            continue
        p["missing"] = 0
        px, liq = cur["price"], cur["liq"] or 0
        p["last"] = px
        prices[key] = px
        p["peak"] = max(p["peak"], px)
        chg = (px / p["avg"] - 1) * 100
        peak_gain = (p["peak"] / p["avg"] - 1) * 100
        drop = (1 - px / p["peak"]) * 100
        held_h = (now - datetime.fromisoformat(p["bought"])).total_seconds() / 36
        if p["liq"] and liq < RUG_LIQ_FRACTION * p["liq"]:
            sell(s, key, px, liq, now, "rug", f"liquidity collapsed to ${liq:,.0f
        elif chg <= -STOP_PCT:
            sell(s, key, px, liq, now, "stop loss", f"price {chg:+.1f}% from entr
        elif peak_gain >= TRAIL_START_PCT and drop >= TRAIL_DROP_PCT:
            sell(s, key, px, liq, now, "trailing stop", f"fell {drop:.1f}% from i
        elif held_h >= MAX_HOURS:
            sell(s, key, px, liq, now, "time limit", f"held {held_h:.0f}h")


def reject_reason(c, age):
    """Which filter stops this coin first (same order as the funnel). None = it p
    if age is None or not (MIN_AGE_MIN <= age <= MAX_AGE_H * 60):
        return "young_old"
    if (c["liq"] or 0) < MIN_LIQ:
        return "low_liq"
    if (c["vol1h"] or 0) < MIN_H1_VOL:
        return "low_vol"
    if not (MIN_FDV <= (c["fdv"] or 0) <= MAX_FDV):
        return "bad_fdv"
    if c["chg1h"] is None or c["chg1h"] < MIN_H1_CHG:
        return "weak_mom"
    if c["chg1h"] > MAX_H1_CHG:
        return "overheated"
    if (c["buyers"] or 0) < MIN_BUYERS:
        return "few_buyers"
    if (c["sellers"] or 0) < MIN_SELLERS or (c["sells"] or 0) == 0 or (c["buys"]
        return "sell_pressure"
    return None
```

```
def shadow_register(s, passed, now, key="shadow", cap=SHADOW_MAX, per_run=None, r
    """Remember every coin that passes the filters (bought or not) so we can see
    sh = s.setdefault(key, {})
    added = 0
    for c in passed:
        if c["pool"] in sh or len(sh) >= cap or (per_run is not None and added >=
            continue
        added += 1
        sh[c["pool"]] = {"reason": (reasons or {}).get(c["pool"], "passed"), "nam
                         "vol1h": c["vol1h"], "chg1h": c["chg1h"], "buyers": c["b
                         "bought": 0, "done": []}


def shadow_step(s, now, get=http_json, key="shadow", path=SHADOW, with_reason=Fal
    """Log the price of shadow coins at each checkpoint (1h, 4h, 24h). Only ever
    sh = s.get(key) or {}
    due = []
    for k, e in sh.items():
        hrs = (now - datetime.fromisoformat(e["t"])).total_seconds() / 3600
        if any(h <= hrs and h not in e["done"] for h in CHECKPOINTS_H):
            due.append(k)
    if not due:
        return 0
    got = fetch_held(due[:limit], get)
    logged = 0
    for k in due[:limit]:
        e = sh[k]
        hrs = (now - datetime.fromisoformat(e["t"])).total_seconds() / 3600
        cur = got.get(k)
        for h in CHECKPOINTS_H:
            if h > hrs or h in e["done"]:
                continue
            if cur is None and hrs < h + 2:
                continue                  # one missed look is not a dead coin: t
            px = cur["price"] if cur else 0.0
            append_csv(path, ["time_utc", "name", "pool", "checkpoint_h", "entry_
                              "chg1h_at_entry", "vol1h", "buyers", "age_min", "
                       [now.isoformat(timespec="seconds"), e["name"], k, h, f"{e[
                        round((px / e["entry"] - 1) * 100, 2) if e["entry"] else
                        round(e["chg1h"] or 0, 1), round(e["vol1h"] or 0), int(e[
                        round(e["age"]) if e["age"] is not None else "", e["bough
            e["done"].append(h)
            logged += 1
    for k in [k for k, e in sh.items() if len(e["done"]) == len(CHECKPOINTS_H)]:
        del sh[k]
    return logged
```

```
def run_once(get=http_json, rug=rug_ok, now=None, pause=True):
    now = now or datetime.now(timezone.utc)
    s = load_state() or {"cash": START_CASH, "positions": {}, "cooldown": [], "be
    sp = sol_price(get)
    if s["bench"] is None and sp:
        s["bench"] = {"sol_qty": START_CASH / sp}
    fresh, raw_first, answered = fetch_candidates(get)
    if raw_first is not None and not os.path.exists(PROBE):
        with open(PROBE, "w") as f:
            json.dump(raw_first, f, indent=1)
    # remember every coin we see, then re-check the ones that have had time to pr
    watch = s.setdefault("watch", {})
    for k, c in fresh.items():
        if k not in watch:
            a0 = age_minutes(c["created"], now)
            watch[k] = (now - timedelta(minutes=a0)).isoformat(timespec="seconds"
    for k in list(watch):
        a0 = (now - datetime.fromisoformat(watch[k])).total_seconds() / 60
        if a0 > MAX_AGE_H * 60 or (len(watch) > WATCH_MAX and a0 > 180):
            del watch[k]
    mature = sorted((k for k in watch if k not in fresh and k not in s["positions
                     and (now - datetime.fromisoformat(watch[k])).total_seconds()
                    key=lambda k: watch[k], reverse=True)[:REFRESH_PER_RUN]
    got = fetch_held(list(s["positions"]) + mature, get) if (s["positions"] or ma
    held = {k: v for k, v in got.items() if k in s["positions"]}
    cands = {**fresh, **{k: v for k, v in got.items() if k not in s["positions"]}
    if not answered:
        print("no data this run (feed unreachable), stopping safely (nothing trad
        return
    prices = {}
    check_exits(s, held, cands, prices, now, count_missing=True)
    # ---- the funnel: why coins are rejected ----
    ages = [a1 for a1 in (age_minutes(c["created"], now) for c in cands.values())
    if ages:
        print(f"ages of coins seen: youngest {min(ages):.0f} min, oldest {max(age
    f = dict(scanned=len(cands), young_old=0, low_liq=0, low_vol=0, bad_fdv=0, we
             sell_pressure=0, owned_or_cooldown=0, rug_check=0, passed=0, bought=
    passed = []
    for key, c in cands.items():
        age = age_minutes(c["created"], now)
        if key in s["positions"] or key in s["cooldown"]:
            f["owned_or_cooldown"] += 1
        elif age is None or not (MIN_AGE_MIN <= age <= MAX_AGE_H * 60):
            f["young_old"] += 1
        elif (c["liq"] or 0) < MIN_LIQ:
```

```
            f["low_liq"] += 1
        elif (c["vol1h"] or 0) < MIN_H1_VOL:
            f["low_vol"] += 1
        elif not (MIN_FDV <= (c["fdv"] or 0) <= MAX_FDV):
            f["bad_fdv"] += 1
        elif c["chg1h"] is None or c["chg1h"] < MIN_H1_CHG:
            f["weak_mom"] += 1
        elif c["chg1h"] > MAX_H1_CHG:
            f["overheated"] += 1
        elif (c["buyers"] or 0) < MIN_BUYERS:
            f["few_buyers"] += 1
        elif (c["sellers"] or 0) < MIN_SELLERS or (c["sells"] or 0) == 0 or (c["b
            f["sell_pressure"] += 1
        else:
            passed.append(c)
    passed.sort(key=lambda c: c["chg1h"], reverse=True)
    f["passed"] = len(passed)
    shadow_register(s, passed, now)
    # wide log: every real-looking coin old enough to judge, with the reason it w
    wide, reasons = [], {}
    for key, c in cands.items():
        if key in s["positions"]:
            continue
        a1 = age_minutes(c["created"], now)
        if a1 is None or not (MIN_AGE_MIN <= a1 <= MAX_AGE_H * 60) or (c["liq"] o
            continue
        wide.append(c)
        reasons[key] = reject_reason(c, a1) or "passed"
    shadow_register(s, wide, now, key="shadow_all", cap=SHADOW_ALL_MAX, per_run=A
    for c in passed[:8]:
        if len(s["positions"]) >= MAX_POSITIONS or f["bought"] >= MAX_BUYS_PER_RU
            break
        ok, why = rug(c["mint"], get) if rug is rug_ok else rug(c["mint"])
        if not ok:
            f["rug_check"] += 1
            continue
        total = equity(s, prices)
        usd = min(total * SIZE_PCT, s["cash"])
        if usd < 10:
            break
        pct = cost_pct(usd, c["liq"])
        fee = usd * pct / 100 + NETWORK_FEE_USD
        qty = (usd - fee) / c["price"]
        s["cash"] -= usd
        s["positions"][c["pool"]] = {"name": c["name"], "qty": qty, "avg": c["pri
                                     "liq": c["liq"], "bought": now.isoformat(tim
        prices[c["pool"]] = c["price"]
```

```
            log_trade(now, c["name"], "buy", qty, c["price"], fee, usd, "momentum",
                      f"+{c['chg1h']:.0f}% in 1h, liquidity ${c['liq']:,.0f}, {int(c[
            f["bought"] += 1
            s["shadow"].get(c["pool"], {})["bought"] = 1
            s.get("shadow_all", {}).get(c["pool"], {})["bought"] = 1
        s["cooldown"] = s["cooldown"][-500:]
        shadow_step(s, now, get)
        shadow_step(s, now, get, key="shadow_all", path=SHADOW_ALL, with_reason=True,
        eq = equity(s, prices)
        hold = s["bench"]["sol_qty"] * sp if (s["bench"] and sp) else ""
        append_csv(EQUITY, ["time_utc", "equity_usd", "cash_usd", "btc_price", "btc_h
                   [now.isoformat(timespec="seconds"), round(eq, 2), round(s["cash"],
                    round(hold, 2) if hold != "" else "", round((eq / hold - 1) * 100
        append_csv(FUNNEL, ["time_utc"] + list(f), [now.isoformat(timespec="seconds")
        with open(STATE, "w") as fh:
            json.dump(s, fh, indent=1)
        print(f"scanned {f['scanned']} | passed filters {f['passed']} | bought {f['bo
              + (f" vs SOL hold ${hold:,.2f}" if hold != "" else ""))
        print("rejected: " + ", ".join(f"{k}={v}" for k, v in f.items() if k not in (


def guard_check(get=http_json, now=None):
    """Fast look at the open positions only: apply the exit rules, save, trade no
    now = now or datetime.now(timezone.utc)
    s = load_state()
    if not s or not s["positions"]:
        return 0
    got = fetch_held(list(s["positions"]), get)
    if got:                                     # feed down = skip this look, never
        prices = {}
        check_exits(s, got, {}, prices, now, count_missing=False)
        with open(STATE, "w") as fh:
            json.dump(s, fh, indent=1)
    return len(s["positions"])


def push_files():
    """Inside GitHub Actions only: commit the data files so Atlas stays fresh and
    if os.environ.get("SAVE_EACH_SCAN") != "1":
        return
    try:
        have = [f for f in (STATE, TRADES, EQUITY, FUNNEL, PROBE, SHADOW, SHADOW_
        for cmd in (["git", "add"] + have, ["git", "commit", "-m", "meme paper ru
            subprocess.run(cmd, check=False, capture_output=True, timeout=60)
    except Exception as e:
        print(f"save skipped ({e})")
```

```
def run_loop(minutes, scan_every=15, look_every=30, get=http_json, rug=rug_ok):
    """Stay awake for `minutes`: a full scan every `scan_every` minutes, a quick
    end = time.time() + minutes * 60
    next_scan = 0.0
    while time.time() < end:
        t0 = time.time()
        if t0 >= next_scan:
            run_once(get=get, rug=rug)
            push_files()
            next_scan = t0 + scan_every * 60
        else:
            guard_check(get)
        time.sleep(max(1.0, look_every - (time.time() - t0)))


if __name__ == "__main__":
    mins = float(os.environ.get("LOOP_MINUTES", "0") or 0)
    if mins > 0:
        run_loop(mins)
    else:
        run_once()
```